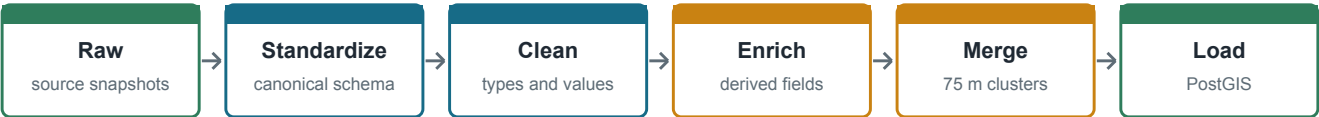


Data Cleaning and Data Merging Methodology

Station-source integration and EV reference-data preparation

Purpose

This note records how EVFlow converts heterogeneous source snapshots into a consistent station dataset. It covers field standardization, type cleaning, connector enrichment, spatial record matching, conflict resolution, and the controls required to reproduce the result.



3	3,569	2,931	75 m
station sources	normalized records	candidate clusters	matching radius

Document	DE-02 - Data Cleaning and Data Merging Methodology
Version	1.0
Prepared	24 July 2026
Scope	National station seed and EV model catalogue
Method owner	EVFlow Data Engineering

The figures above describe the current national API seed. A separate Jakarta notebook applies a bounding-box filter before clustering and therefore produces a different station count; the two outputs are not interchangeable.

1. Scope and working principles

EVFlow combines three station sources with different schemas and coverage: PLN SPKLU, Open Charge Map (OCM), and OpenStreetMap (OSM). The EV model catalogue follows a separate cleaning path because it contains vehicle specifications rather than station observations.

Raw data remains evidence

Cleaning is performed after a source snapshot is loaded. The original JSON, CSV, or ZIP artifact is not edited in place. This preserves provenance, supports reruns, and makes transformation defects traceable to either source data or code.

1.1 Source characteristics

Source	Raw structure	Coverage in current snapshot	Primary cleaning concern
PLN SPKLU	Flat JSON records with nested charger boxes	3,029 nationwide records	String-encoded power, invalid coordinates, inconsistent charge labels
Open Charge Map	Nested POI, address, status, and connection objects	527 Greater Jakarta POIs	Nested connectors, nullable metadata, community verification dates
OpenStreetMap	Nodes, ways, relations, and free-form tags	13 Greater Jakarta elements	Sparse tags, alternate identity fields, center points for non-node objects
Kaggle EV catalogue	CSV in ZIP or extracted CSV	60 vehicle-model rows	Free-text units, ranges, decimal commas, duplicate model names

1.2 Canonical station record

Each station observation is converted to a common record before matching. The canonical form removes source-specific nesting while retaining the source identifier needed for lineage.

Field group	Canonical fields	Purpose
Identity	id, source, name, operator	Stable source reference and display identity
Location	latitude, longitude, address, province, city	Spatial search, clustering, and presentation
Charging	power_kw, charge_type, connectors, status	Connector filtering and charging-speed classification
Quality and lineage	date_verified, connector_inferred, sources[]	Freshness context, estimation disclosure, and merged-source traceability

1.3 Processing boundary

The runtime API does not retrieve upstream data. It reads committed snapshots through `api/sources.py`. The seed command calls the loaders, applies normalization and clustering, truncates the station table, and inserts the rebuilt result through `scripts/seed_db.py`.

This is a batch replacement pipeline, not streaming ingestion and not an incremental merge into existing station rows.

2. Station data cleaning

Cleaning converts source-specific values into comparable data types and field meanings. Derived connector attributes are calculated only after the base record has been standardized.

2.1 Source-specific parsing

Rule	PLN	OCM	OSM
Station name	nama_lokasi	AddressInfo.Title	name, name:en, brand, operator, network, or ref
Coordinates	Cast to float; reject invalid, non-finite, and 0,0	Read AddressInfo; reject missing latitude or longitude	Use node coordinates or way/relation center; reject missing values
Power	Read first numeric token from watt	Use per-connection PowerKW and retain the maximum at station level	Parse charging output tags when present
Connector count	total_konektor; default one when absent	Connection.Quantity or one	capacity when numeric; otherwise one
Status	status=1 becomes operational	Map IsOperational to operational or non-operational	Retain access tag as supplied

2.2 Common cleaning rules

Input condition	Action	Reason
Missing station coordinates	Exclude the observation from the station pipeline	The record cannot support spatial search or reliable matching
NaN power	Convert to null	Prevents non-finite values from reaching JSON and database fields
Power with unit text	Extract the leading numeric value and store kW as float	Creates one comparable unit across sources
Blank optional text	Retain as null rather than inventing a value	Preserves the distinction between absent and observed data
Unknown connector evidence	Keep the station but omit connector details	Avoids presenting an unsupported connector type

2.3 Connector enrichment

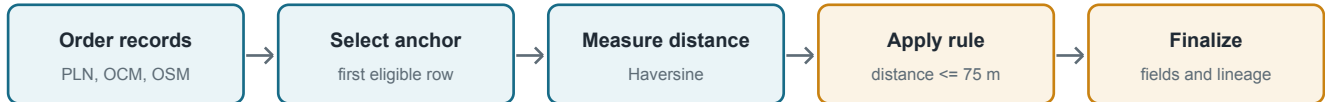
Connector type is not consistently available in the source snapshots. EVFlow therefore applies a deterministic inference rule using observed power and, when power is missing, PLN's charge category.

Condition	Connector result	Speed tier
Power <= 7 kW	AC Type 2	slow
Power > 7 and <= 22 kW	AC Type 2	medium
Power > 22 and <= 50 kW	CCS2	medium
Power > 50 and <= 150 kW	CCS2	fast
Power > 150 kW	CCS2	ultra_fast
No power; slow or medium label	AC Type 2	Use normalized source label
No power; fast or ultra-fast label	CCS2	Use normalized source label

Method note. Parsing coordinates, units, nulls, and status values is data cleaning. Connector inference is feature engineering and is marked with connector_inferred=true so downstream users can distinguish estimates from source-reported evidence.

3. Spatial data merging

A physical charging site may be represented independently by PLN, OCM, and OSM. EVFlow resolves these records through deterministic, proximity-based clustering. The operation is record linkage: it creates one station entity while retaining all contributing source names.



Match Merge into the anchor cluster; retain source lineage.

No match Create a new cluster using the current record as its anchor.

3.1 Matching rule

Records are sorted by source priority and source ID. PLN has priority 0, OCM priority 1, and OSM priority 2. Each record is compared with existing cluster anchors using Haversine distance. A distance of 75 metres or less is treated as a match; otherwise a new cluster is created.

```

ordered = sort(records, source_priority, id)
for record in ordered:
    anchor = first cluster within 75 m
    if anchor exists: merge(record, anchor)
    else: create_cluster(record)
  
```

3.2 Conflict-resolution policy

Attribute	Resolution rule	Effect
Cluster ID and coordinates	Retain the anchor record	A PLN record anchors the cluster when one is present
Name, address, region, operator	Keep the first non-empty value in source-priority order	Lower-priority sources fill gaps but do not overwrite populated PLN fields
Source lineage	Append each distinct source to sources[]	The merged record remains traceable to PLN, OCM, and/or OSM
Connector identity	Group by connector type and power_kw	Retains the union of distinct connector configurations
Connector count	Use maximum count, not sum	Reduces double-counting when two sources describe the same hardware
Station power and speed	Recalculate from the highest-powered merged connector	Keeps station-level filters consistent with the final connector list

3.3 Determinism and output

The ordering rule makes the result independent of input list order. Given the same snapshots, radius, and code revision, clustering produces the same anchor IDs and field precedence. In the current national seed, 3,569 normalized station observations produce 2,931 merged station records.

Count interpretation

3,569 is a source-record count and 2,931 is a merged-record count. Neither number proves that every listed site is currently operational. Source scope, snapshot date, and the matching rule all affect the result.

4. EV catalogue cleaning

The 60-row EV catalogue is prepared independently from station data. Its purpose is to provide normalized battery and range values for range-aware routing, not to estimate vehicle ownership or market share.

Source value	Transformation	Output example
Vehicle Name	Trim whitespace; reject blank names; generate a lowercase slug ID	Hyundai Ioniq 5 -> hyundai-ioniq-5
Make and model	Use the first name token as make and the remaining tokens as model	Hyundai Ioniq 5
Battery Capacity	Extract numeric values; convert decimal commas to decimal points	26.7 kWh -> 26.7
Range (Jarak Tempuh)	Extract all numeric values and retain the minimum	200 - 300 km -> 200
Price, charging time, source URL	Trim whitespace; convert blanks to null	Preserved as descriptive text
Duplicate model slug	Keep the first parsed row for each slug	One catalogue entry per normalized ID

Conservative range assumption

When a published range is expressed as an interval, EVFlow retains the lower bound. Runtime reachability then multiplies full range by state of charge and a 0.85 safety factor. The lower-bound parse is a cleaning decision; the safety factor is a routing-model assumption.

5. Validation and quality controls

Unit tests cover normalization, connector inference, connector aggregation, source-priority clustering, deterministic anchors, and seed output. The table below separates implemented checks from controls still needed for a production refresh process.

Control	Current status	Operational interpretation
Missing coordinates	Implemented	All source loaders exclude records that cannot provide both coordinates
PLN finite and 0,0 coordinates	Implemented	Invalid or placeholder PLN positions are rejected
OCM/OSM geographic bounds and finite values	Gap	Presence is checked, but complete numeric and study-area validation should be added
Connector and speed-tier rules	Unit tested	Boundary values and unknown signals have deterministic outcomes
75 m clustering and source precedence	Unit tested	Merge behavior is stable for the same input
Schema-drift detection	Gap	A changed upstream payload can silently reduce field coverage
Refresh manifest and checksum	Gap	Retrieval metadata is not yet recorded as a complete machine-readable manifest
Ambiguous-cluster review queue	Gap	Dense-site matches are not currently escalated for manual review

6. Limitations and operating controls

The current method is appropriate for a small, periodically refreshed station inventory, but its assumptions must remain visible when the output is used in analysis or product decisions.

6.1 Known limitations

Limitation	Consequence	Required treatment
Proximity-only matching	Two different sites within 75 m may be merged, particularly in malls, rest areas, or dense urban sites	Review ambiguous clusters using name, address, operator, and connector similarity
Anchor coordinates are not averaged	The merged position inherits the highest-priority source coordinate	Treat the anchor as source-preferred, not survey-grade ground truth
Connector type is inferred	AC Type 2 and CCS2 may not reflect mixed or unusual installations	Expose connector_inferred and replace estimates when verified evidence becomes available
Connector counts use maximum	The result may undercount genuinely different hardware with the same type and power	Retain source records for audit and reconcile verified inventories separately
Source scopes differ	PLN is national while the current OCM and OSM snapshots cover Greater Jakarta	Apply a common geographic filter before making source-to-source coverage comparisons
Destructive station seed	A failed or partial refresh can replace a valid station table	Validate in staging, back up, load transactionally, and verify row counts before release

6.2 Refresh checklist

Before cleaning	Before merging	Before release
Record retrieval time, source identifier, query, licence, row count, and SHA-256 checksum	Compare null rates, coordinate bounds, power distributions, and source counts with the prior snapshot	Review cluster-count change, sample ambiguous matches, and confirm source lineage coverage
Store a dated raw artifact without replacing the prior snapshot	Run source, connector, deduplication, and seed-build unit tests	Back up the database, load in a transaction, and run post-load spatial/API checks

6.3 Implementation traceability

Responsibility	Implementation	Verification
Source parsing and canonical schema	backend-ev-flow/api/sources.py	tests/test_sources.py
Connector inference and aggregation	backend-ev-flow/api/connectors.py	tests/test_connectors.py
Spatial clustering and conflict resolution	backend-ev-flow/api/dedup.py	tests/test_dedup.py
Station rebuild and PostGIS load	backend-ev-flow/scripts/seed_db.py	tests/test_seed_build.py
EV catalogue parsing	backend-ev-flow/api/evmodels.py	EV model unit and API tests

Method statement

EVFlow preserves source snapshots, standardizes them to a canonical schema, records inferred attributes explicitly, and merges station observations through deterministic 75 m anchor-based clustering. The output is reproducible for a fixed set of snapshots and code, subject to the documented assumptions and validation gaps.